Assignment 4

1. Write an SQL query to display the names and salaries of employees whose salary is greater than the average salary in the company. Explain how the subquery works.

```
mysql> select avg(salary ) from employees;
+--------------+
| avg(salary ) |
+--------------+
| 68000.000000 |
+--------------+
1 row in set (0.00 sec)

mysql> select name,salary from employees
    -> where salary >(
    -> select avg(salary ) from employees);
+--------------+----------+
| name         | salary   |
+--------------+----------+
| Neha Verma   | 75000.00 |
| Vikas Rao    | 80000.00 |
| Arjun Nair   | 72000.00 |
| Anjali Gupta | 90000.00 |
| Rohit Jain   | 85000.00 |
+--------------+----------+
5 rows in set (0.00 sec)
```

The subquery (select avg (salary) from employees) calculates the average salary of all employees. Then the outer query selects employees whose salary is greater than that average. The subquery runs first, returns a single value (average salary), and then the outer query compares each employee's salary with that value.

2. Write a query to retrieve the top 5 highest-paid employees from an employees table. Explain how sorting affects the output.

```
mysql> select * from employees
    -> order by salary desc
    -> limit 5;
+--------+--------------+------------+-----------+----------+------------+
| emp_id | name         | department | job_role  | salary   | commission |
+--------+--------------+------------+-----------+----------+------------+
|      9 | Anjali Gupta | IT         | Team Lead | 90000.00 |   10000.00 |
|     10 | Rohit Jain   | Marketing  | Manager   | 85000.00 |       NULL |
|      5 | Vikas Rao    | IT         | Developer | 80000.00 |    7000.00 |
|      2 | Neha Verma   | HR         | Manager   | 75000.00 |       NULL |
|      7 | Arjun Nair   | Finance    | Analyst   | 72000.00 |       NULL |
+--------+--------------+------------+-----------+----------+------------+
5 rows in set (0.00 sec)
```

Order by salary desc sorts salaries from highest to lowest. Limit 5 selects only the top 5 records. Sorting determines which records appear first.

3. Write a query to calculate:
   - Total number of employees
   - Average salary
   - Minimum and maximum salary

   Explain the difference between aggregate and scalar functions.

```
mysql> select count(*) from employees;
+----------+
| count(*) |
+----------+
|       10 |
+----------+
1 row in set (0.00 sec)

mysql> select avg(salary) from employees;
+--------------+
| avg(salary)  |
+--------------+
| 68000.000000 |
+--------------+
1 row in set (0.00 sec)

mysql> select min(salary) from employees;
+-------------+
| min(salary) |
+-------------+
|    48000.00 |
+-------------+
1 row in set (0.01 sec)

mysql> select max(salary) from employees;
+-------------+
| max(salary) |
+-------------+
|    90000.00 |
+-------------+
1 row in set (0.00 sec)
```

Difference Between Aggregate and Scalar Functions:

Aggregate Functions	**Scalar Functions**
Work on multiple rows	Work on single value
Return one result	Return one result per row
Example: COUNT(), AVG(), SUM()	Example: UPPER(), ROUND(), LENGTH()

4. Given a sales table with columns (region, amount), write a query to find total sales per region. Filter only those regions where total sales exceed 50,000.

```
mysql> select region , sum(amount) as total_sales from sales
    -> group by region
    -> having sum(amount) >50000;
+--------+-------------+
| region | total_sales |
+--------+-------------+
| North  |    60000.00 |
| South  |    75000.00 |
| East   |    60000.00 |
| West   |    60000.00 |
+--------+-------------+
4 rows in set (0.02 sec)
```

5. Write a query to find the number of unique job roles in an employees table. Explain why DISTINCT is necessary here.

```
mysql> select count(distinct (job_role) )from employees;
+-----------------------------+
| count(distinct (job_role) ) |
+-----------------------------+
|                           8 |
+-----------------------------+
1 row in set (0.01 sec)
```

Without distinct, duplicate job roles would be counted multiple times. Distinct ensures each job role is counted only once.

6. Write a query to retrieve students who scored between 60 and 80 marks. Rewrite the same query using BETWEEN.

```
mysql> select * from student
    -> where marks >=60 and marks <=80;
+------------+--------+-------+
| student_id | name   | marks |
+------------+--------+-------+
|          2 | Aditya |    72 |
|          4 | Kunal  |    67 |
|          8 | Ishaan |    76 |
|         10 | Dev    |    60 |
+------------+--------+-------+
4 rows in set (0.00 sec)
```

```
mysql>
mysql> select * from student
    -> where marks between 60 and 80;
+------------+--------+-------+
| student_id | name   | marks |
+------------+--------+-------+
|          2 | Aditya |    72 |
|          4 | Kunal  |    67 |
|          8 | Ishaan |    76 |
|         10 | Dev    |    60 |
+------------+--------+-------+
4 rows in set (0.01 sec)
```

7. Write a query to display employees whose commission is NULL. Explain the correct way to check NULL values in SQL.

```
mysql>
mysql> select * from employees where
    -> commission is null;
+--------+------------+------------+------------+----------+------------+
| emp_id | name       | department | job_role   | salary   | commission |
+--------+------------+------------+------------+----------+------------+
|      2 | Neha Verma | HR         | Manager    | 75000.00 |       NULL |
|      4 | Priya Mehta | Finance    | Accountant | 65000.00 |       NULL |
|      7 | Arjun Nair | Finance    | Analyst    | 72000.00 |       NULL |
|     10 | Rohit Jain | Marketing  | Manager    | 85000.00 |       NULL |
+--------+------------+------------+------------+----------+------------+
4 rows in set (0.00 sec)
```

8. Write a query to increase the salary of employees in the “IT” department by 10%. Explain how arithmetic operations are handled in SQL.

```
mysql> update employees
    -> set salary= salary + (salary* 0.10)
    -> where department= "IT";
Query OK, 4 rows affected (0.02 sec)
Rows matched: 4  Changed: 4  Warnings: 0

mysql> select * from employees;
+--------+--------------+------------+------------+----------+------------+
| emp_id | name         | department | job_role   | salary   | commission |
+--------+--------------+------------+------------+----------+------------+
|      1 | Amit Sharma  | IT         | Developer  | 66000.00 |    5000.00 |
|      2 | Neha Verma   | HR         | Manager    | 75000.00 |       NULL |
|      3 | Rahul Singh  | IT         | Tester     | 55000.00 |    3000.00 |
|      4 | Priya Mehta  | Finance    | Accountant | 65000.00 |       NULL |
|      5 | Vikas Rao    | IT         | Developer  | 88000.00 |    7000.00 |
|      6 | Sneha Kapoor | Marketing  | Executive  | 55000.00 |    4000.00 |
|      7 | Arjun Nair   | Finance    | Analyst    | 72000.00 |       NULL |
|      8 | Kiran Patel  | HR         | Recruiter  | 48000.00 |    2000.00 |
|      9 | Anjali Gupta | IT         | Team Lead  | 99000.00 |   10000.00 |
|     10 | Rohit Jain   | Marketing  | Manager    | 85000.00 |       NULL |
+--------+--------------+------------+------------+----------+------------+
10 rows in set (0.00 sec)

mysql>
```

9. Write a query to delete records of students who scored less than 40 marks. What precaution should be taken before executing DELETE?

```
mysql> delete from student
    -> where marks<40;
Query OK, 2 rows affected (0.01 sec)

mysql> select * from student;
+------------+--------+-------+
| student_id | name   | marks |
+------------+--------+-------+
|          1 | Riya   |    85 |
|          2 | Aditya |    72 |
|          4 | Kunal  |    67 |
|          5 | Megha  |    91 |
|          6 | Varun  |    58 |
|          7 | Tanya  |    44 |
|          8 | Ishaan |    76 |
|         10 | Dev    |    60 |
+------------+--------+-------+
8 rows in set (0.00 sec)

mysql>
```

precaution should be taken before executing DELETE--

Always run a SELECT first to check affected rows.

10.Write a query to find employees who earn more than the average salary of their department (without using joins). Explain the logic of the subquery.

```
mysql> select * from employees
    -> where salary> (
    -> select avg(salary) from employees
    -> where department= employees.department );
+--------+--------------+------------+-----------+----------+------------+
| emp_id | name         | department | job_role  | salary   | commission |
+--------+--------------+------------+-----------+----------+------------+
|      2 | Neha Verma   | HR         | Manager   | 75000.00 |       NULL |
|      5 | Vikas Rao    | IT         | Developer | 88000.00 |    7000.00 |
|      7 | Arjun Nair   | Finance    | Analyst   | 72000.00 |       NULL |
|      9 | Anjali Gupta | IT         | Team Lead | 99000.00 |   10000.00 |
|     10 | Rohit Jain   | Marketing  | Manager   | 85000.00 |       NULL |
+--------+--------------+------------+-----------+----------+------------+
5 rows in set (0.01 sec)
```

IN this subquery, for each employee, the subquery calculates the average salary of that employee's department. Then it compares the employee's salary with their department's average. The subquery runs once for each row in the outer query.